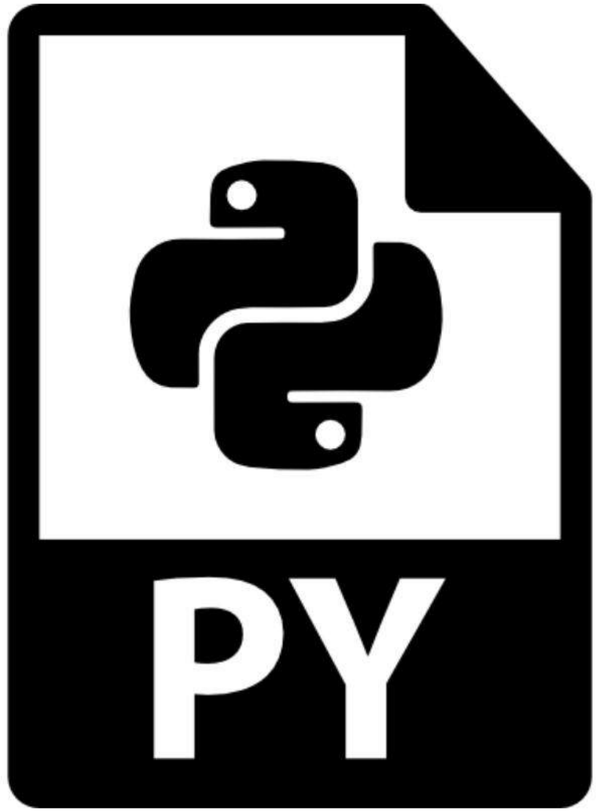
PYTHON TUTORIAL

Uppercase Text Converter

Mastering string manipulation and file handling in Python.

Prepared for Python Learners



The Challenge

Goal

Write a Python program that reads `text.txt` and creates `uppercase.txt`.

- **Convert Case:** Every line must be the UPPERCASE version of the original.
- **Preserve Structure:** Maintain the exact same number of lines.
- **Clean Output:** No extra blank lines between the text.

Input vs. Output

text.txt (Input)

```
Hello World
Python Programming
File Handling
```

.upper()

uppercase.txt (Output)

```
HELLO WORLD
PYTHON PROGRAMMING
FILE HANDLING
```

Key Concept: String Methods

The `.upper()` Method

Python strings have a built-in method that returns a copy of the string with all alphabetic characters converted to uppercase.

- **Characters:** 'a' becomes 'A'.
- **Ignored:** Numbers, punctuation, and whitespace (like `\n`) are unchanged.
- **Non-destructive:** It returns a *new* string; it doesn't change the original.

```
# Example

original = "Hello World\n"
modified = original.upper()

# Result: "HELLO WORLD\n"
```

Solution 1: Iterative Approach

How it works

We process the file **one line at a time**. We read a line, immediately capitalize it, write it to the new file, and then forget it before moving to the next line.

Why use it?

This is the most **memory-efficient** way. You can process a 10GB text file using only a few kilobytes of RAM because you never load the whole file at once.

Flowchart: Iterative Approach

Open Files

'r' mode (in)
'w' mode (out)

Read Line

Get next string

Transform

Apply `.upper()`

Write

Save to new file

Repeat

Until EOF

Code: Iterative Solution

```
def convert_to_uppercase():
    try:
        with open('text.txt', 'r') as infile, \
             open('uppercase.txt', 'w') as outfile:

            # Iterate over the file object directly
            for line in infile:
                # Convert to upper and write immediately
                outfile.write(line.upper())

        print("Conversion successful.")

    except FileNotFoundError:
        print("Source file not found.")

convert_to_uppercase()
```

Code Breakdown

- `with open( ... )`: Ensures files are safely closed.
- `for line in infile`: The most Pythonic way to read files.
- `line.upper()`: Transforms "Hello\n" to "HELLO\n".
- `outfile.write()`: Writes the transformed string.

Solution 2: Bulk Processing

How it works

Instead of looping line-by-line, we read the **entire file** into a single string variable, transform the whole block of text at once, and write it back.

Why use it?

This is extremely concise. For small-to-medium files (e.g., config files, short logs), this is often the fastest and cleanest code to write.

One-Shot Process

Flowchart: Bulk Processing

1. Read All

Load the entire `text.txt` content into a single variable using `read()`.

2. Transform All

Apply `.upper()` to the massive string. All lines change at once.

3. Write All

Dump the huge uppercase string into `uppercase.txt` in one go.

Code: Bulk Solution

```
def convert_fast():
    try:
        with open('text.txt', 'r') as f:
            content = f.read()  # Read everything

        # Transform and write in one block
        with open('uppercase.txt', 'w') as f:
            f.write(content.upper())

        print("Done!")

    except FileNotFoundError:
        print("File missing.")

convert_fast()
```

Code Breakdown

- `f.read()`: Reads the whole file, including newlines, into one string.
- `content.upper()`: Converts every character in that massive string.
- `write()`: Does not add extra newlines; it writes exactly what you give it.

Handling Newlines

"No Extra Blank Lines"

The problem specifically asks to preserve structure without adding extra gaps. This is why we use `file.write()` instead of `print()`.

- **Input Line:** `"Hello\n"`
- **After `.upper()`:** `"HELLO\n"`
- **Using `write()`:** Writes "HELLO" + newline. Perfect.
- **Using `print()`:** Would write "HELLO" + newline + *another* newline. (Bad)

Common Mistake

```
outfile.write(line.upper() + "\n")
```

This adds a double newline because the original line *already* has one!

Comparing Approaches

Feature	Solution 1 (Iterative)	Solution 2 (Bulk Read)
Memory Usage	**Minimal** (Constant)	**High** (Depends on file size)
Speed (Small Files)	Fast	**Fastest**
Speed (Huge Files)	**Reliable**	May crash (Memory Error)
Code Simplicity	Standard Loop	Very Concise